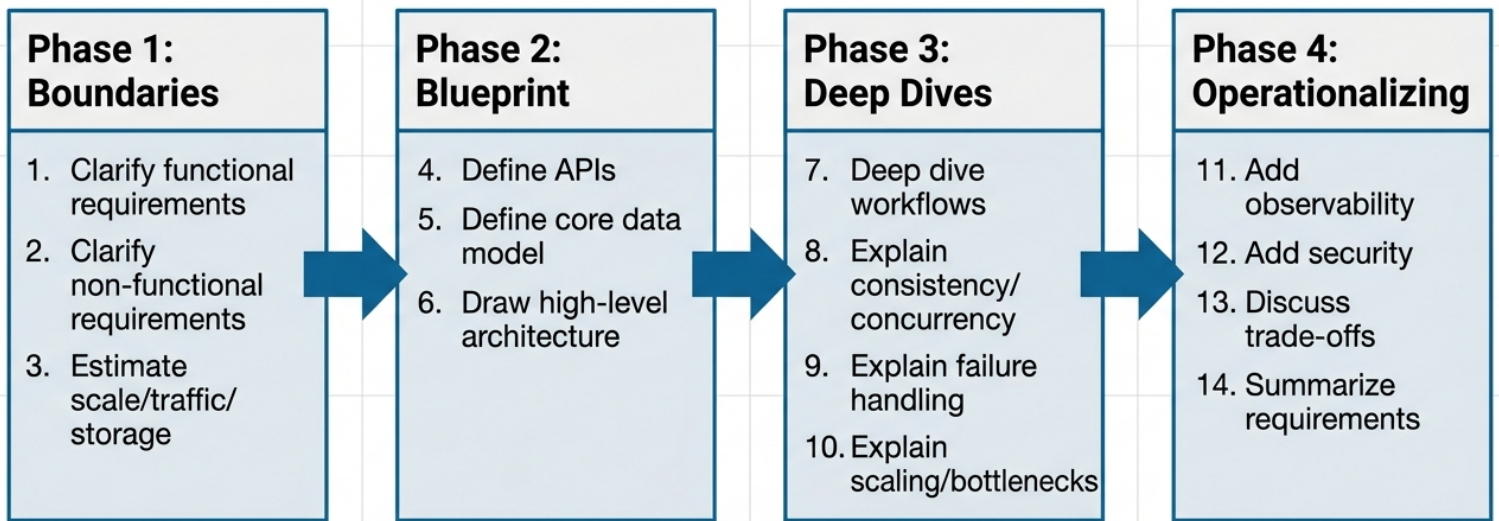


# System Design Playbook: Distributed Railway Reservation

An architect-level deconstruction of concurrency, consistency, and scale.

> STATUS: INTERVIEW\_PREP\_INITIALIZED  
> ENV: FAANG\_BAR\_RAISER



// ARCHITECT MINDSET

For every component, explain **why** it exists, how it **scales**, what happens when it fails, and the **trade-off** accepted.

## Functional Boundaries

- Search
- Availability
- Reservation
- Booking
- Payment
- Confirmation
- Cancellation
- Notifications
- History
- Admin

**CORE INVARIANT:** Guarantee that a seat is never confirmed for two passengers.

## Non-Functional Targets



**Availability:** 99.99% for critical booking.



**Performance:** Search p95 < 500 ms.



**Scalability:** Horizontal scaling of stateless services.



**Fault Tolerance:** Multi-AZ, retries, circuit breakers.



**Disaster Recovery:** Defined RPO/RTO.

```
> NORMAL_TRAFFIC: 1,000 req/sec  
> PEAK_BURST: 100,000 concurrent req/sec  
> INTERVIEW_TIP: Explicitly state these are  
illustrative assumptions. Ask the interviewer for  
actual numbers to prove collaborative design.
```

**Eventual  
Consistency**

## The Consistency Spectrum

**Strong  
Consistency**

### Read-Heavy Paths

- Search
- Analytics
- Notifications

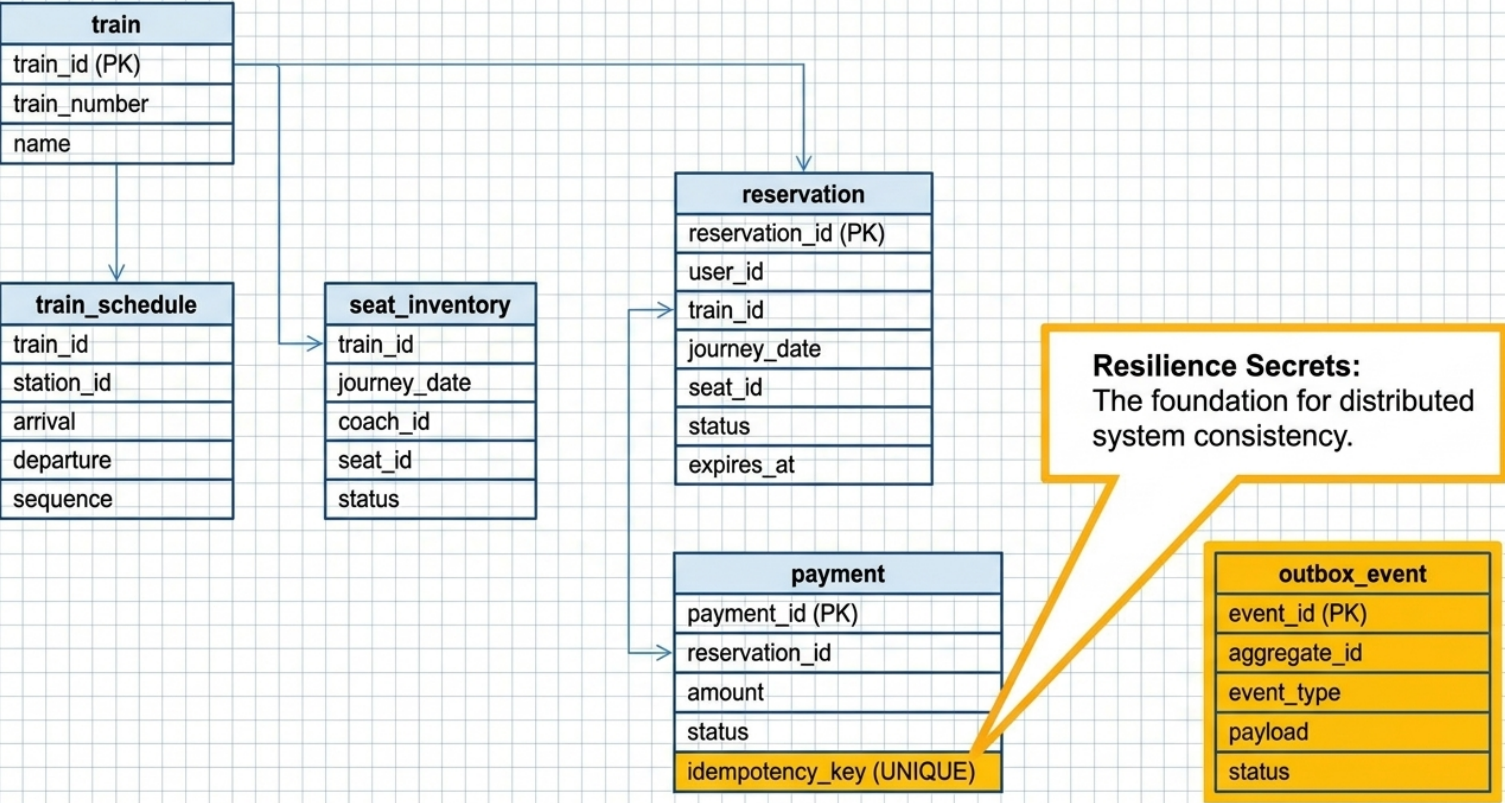
**High Cache Dependency.**  
Stale data acceptable for a few seconds.

### Write-Heavy Paths

- Seat Allocation
- Confirmed Bookings
- Payments

**ACID Dependency.**  
Strict transactional boundaries.





# Boundary Definition: The API Layer

## Read Operations

GET /trains?from=MAS&to=SBC&date=2026-08-20

GET /trains/{trainId}/availability?date=2026-08-20&class=3A

## Write Operations (Transactional)

POST /reservations

Body: { 'trainId': 12623, 'journeyDate': '2026-08-20', 'class': '3A', 'passengers': [...] } }

## Idempotent Mutations

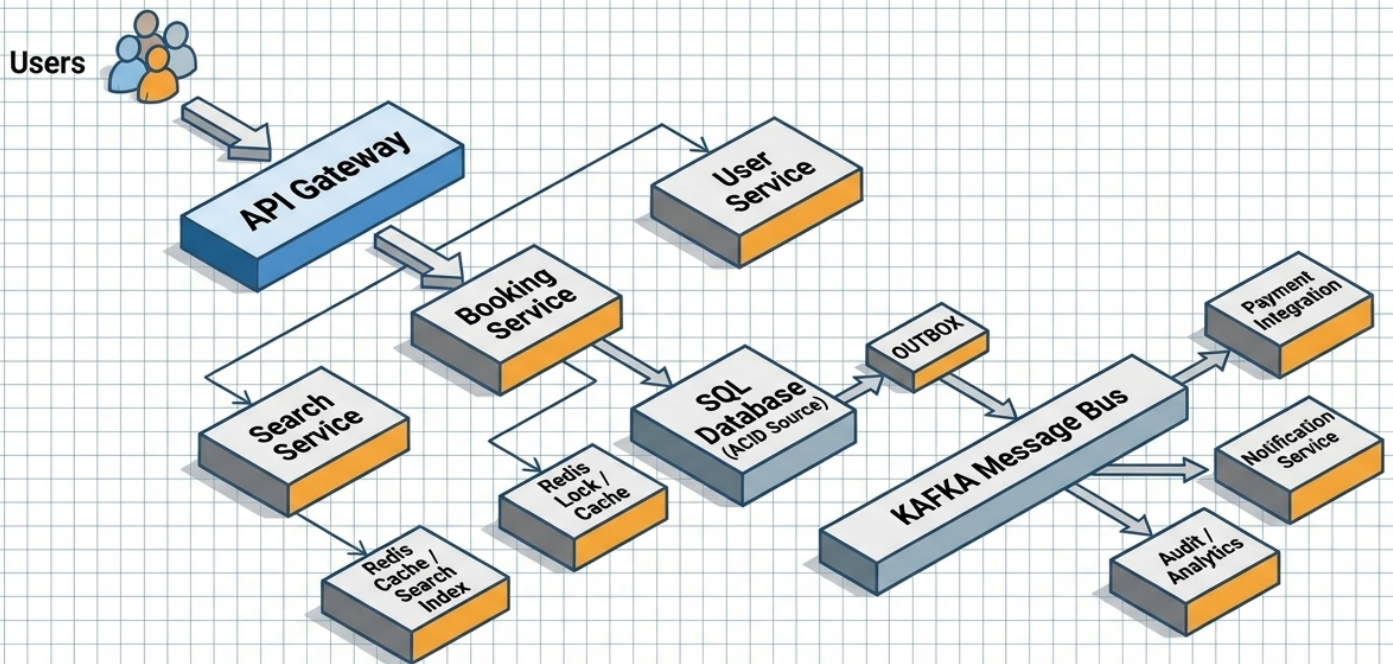
POST /payments

Headers: Idempotency-Key: <unique-key>

POST /reservations/{id}/confirm

POST /reservations/{id}/cancel

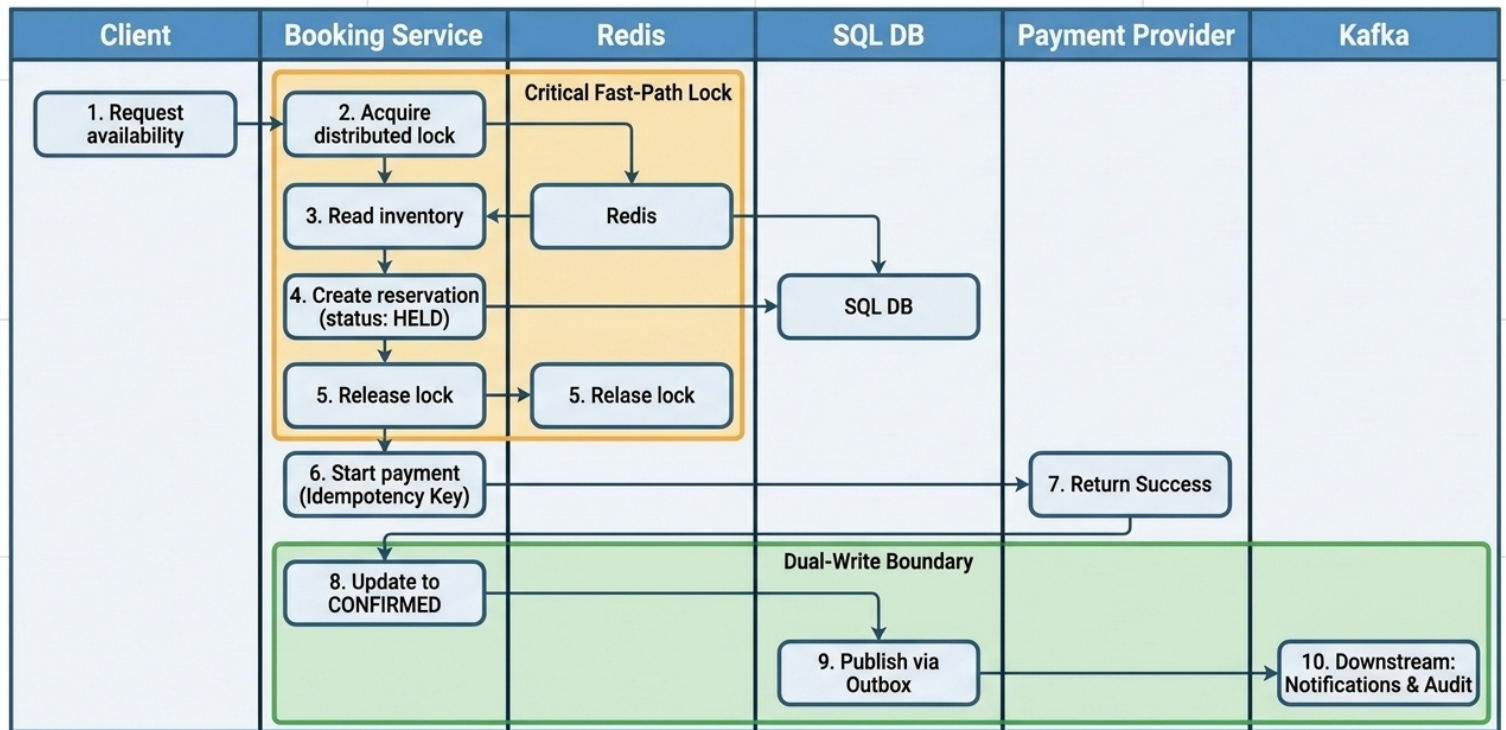
**State Mutations** must be idempotent to survive network partitions and client retries.



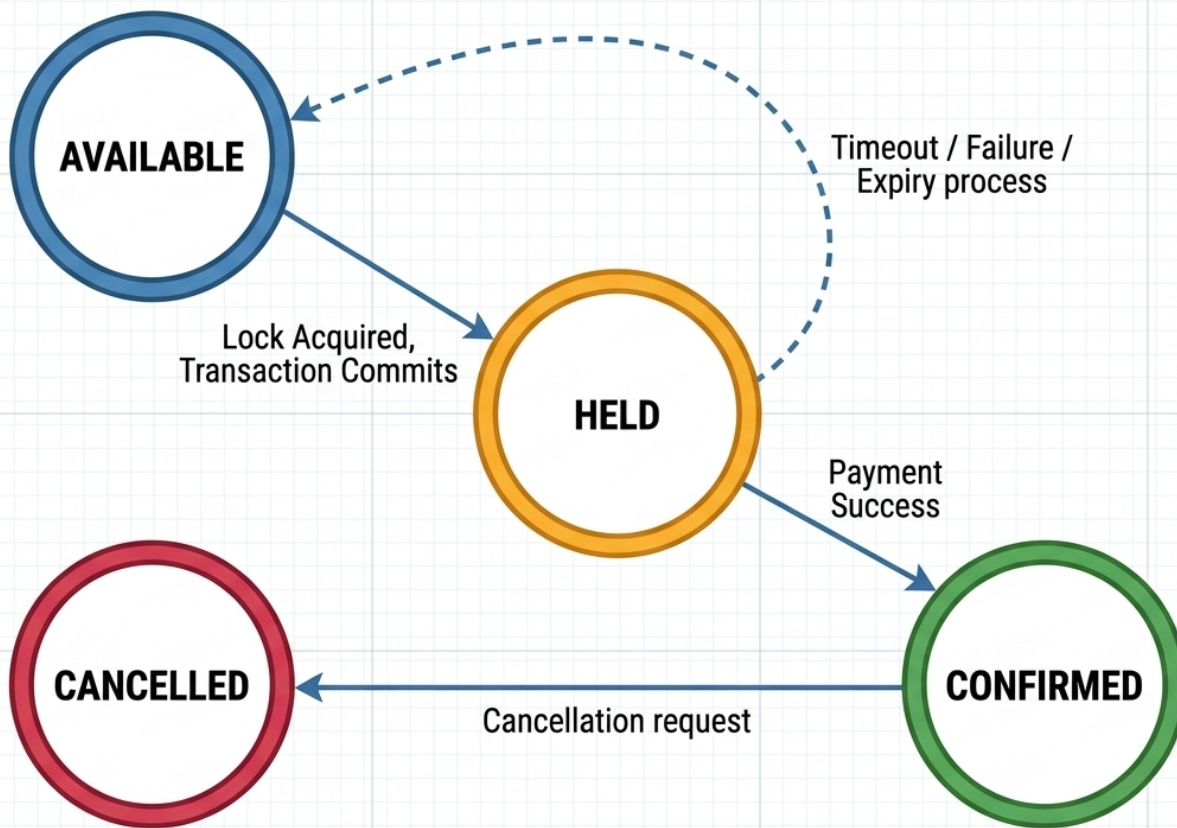
**Key Insight:** Avoid making every request hit the primary database. Cache read-heavy data, partition hot inventory, and keep the strongly consistent write path small.



# Transaction Flow: Reservation & Payment







// INTERVIEW TIP:  
Do not hold the  
Redis lock while  
the user waits  
for payment.

Hold the database  
state instead.

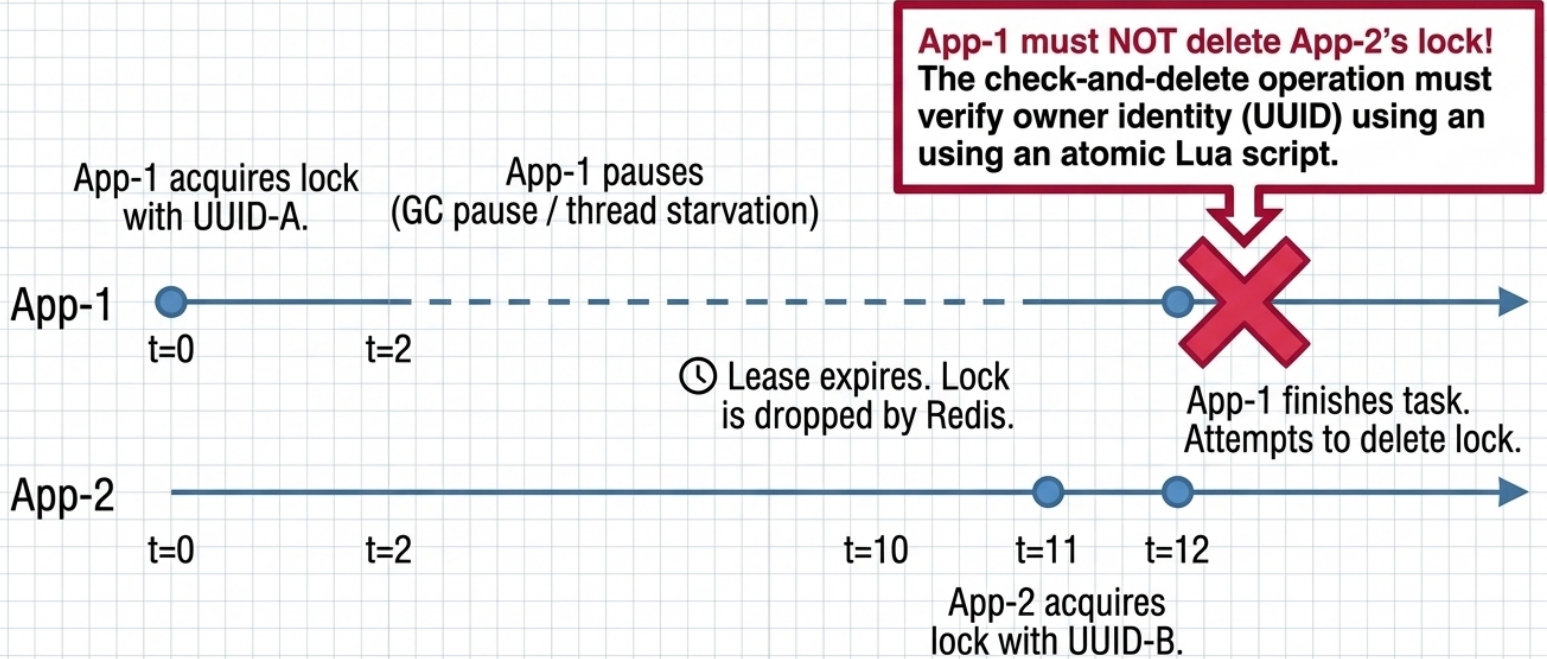
# The Concurrency Matrix: Solving Double Booking

|             | JVM Synchronized Lock        | Redis Distributed Lock                      | Relational DB (ACID)              |
|-------------|------------------------------|---------------------------------------------|-----------------------------------|
| Scope       | Local JVM only               | Distributed across cluster                  | Distributed, system of record     |
| Speed       | Nanoseconds                  | Milliseconds                                | Tens of milliseconds              |
| Reliability | Fails in multi-node setup    | Can suffer from lease expiry or split-brain | Highest, survives app crashes     |
| Verdict     | Insufficient for distributed | Excellent for reducing DB contention        | The ultimate correctness boundary |

**A JVM lock fails because a synchronized block on App-1 does not lock threads on App-2. Coordination must leave the application layer.**

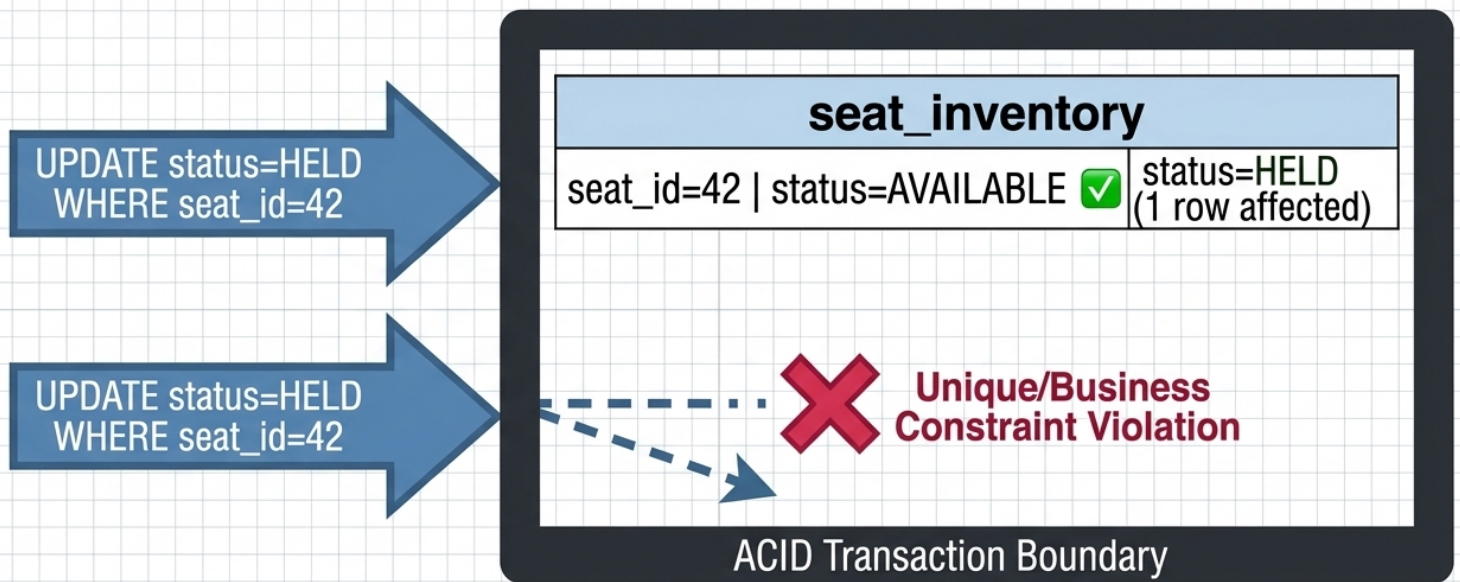
# The Distributed Race Condition: Lease Expiry and Stale Locks

```
SET lock:train:12623:2026-08-20:3A  
<UUID> NX PX 10000
```



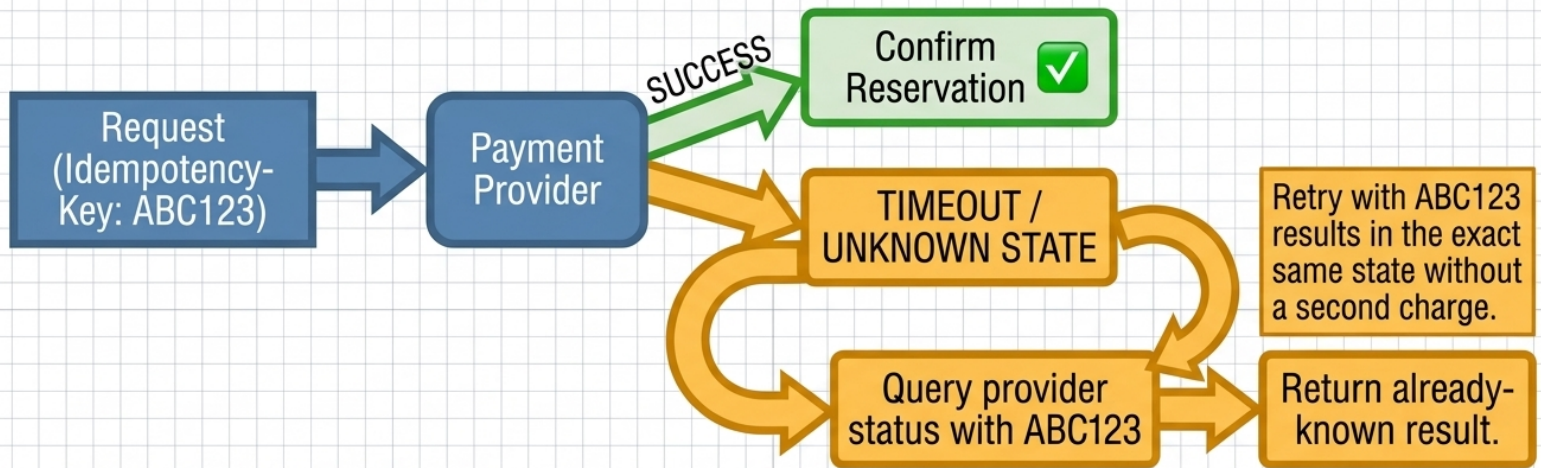


# Database Remains the Final Correctness Boundary



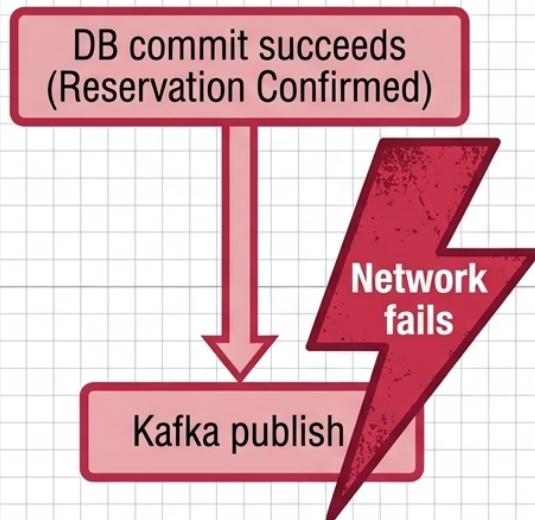
**Even if distributed coordination fails completely, the database schema physically prevents two conflicting confirmed allocations.**

# Idempotency and Payment Boundaries



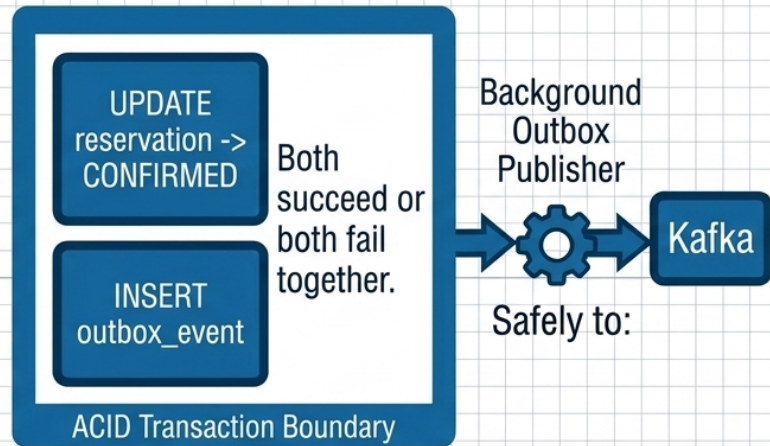
**Rule:** Never assume a timeout means payment failed. External payment systems create distributed transaction boundaries. Safe retries require stable request identity.

## The Dual-Write Trap



**State divergence.  
Customer paid, but no  
ticket generated.**

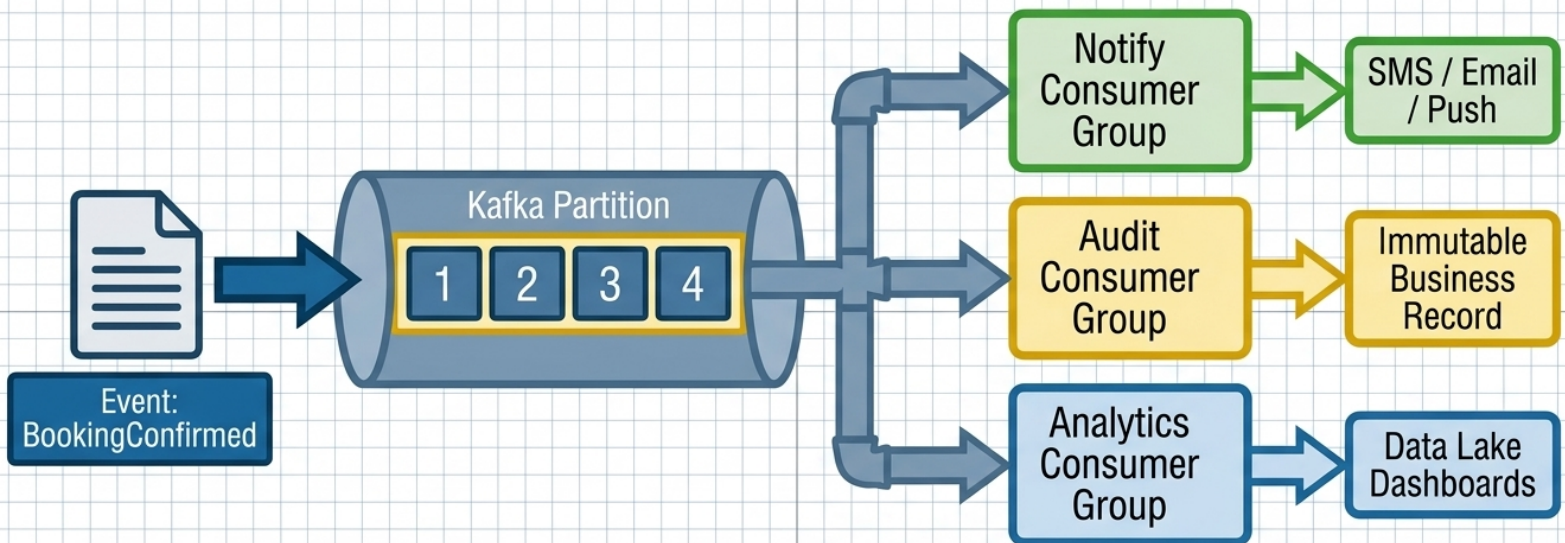
## The Outbox Pattern



**The DB state change and the event  
must not diverge. Write both to  
the DB in a single transaction.**



# Event Fan-Out and Ordering



**> USE CASE CONSTRAINT:** Use a stable partition key (e.g., reservation\_id) for entities that require ordered processing to ensure events for the same seat aren't processed out of sequence by different consumers.

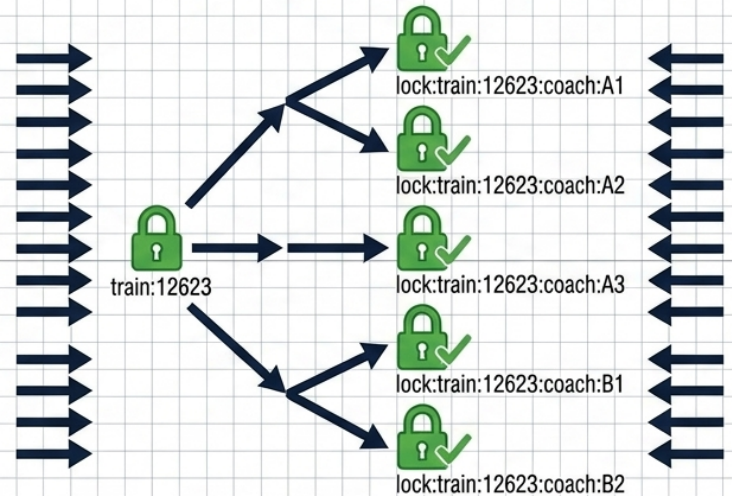
# Scale by Partitioning

## Anti-Pattern



Avoid global train locks.  
Creates a massive hot spot.

## Optimized Pattern



Partitioning must match the allocation algorithm. Fine-grained locking (per seat) is great for throughput, but creates deadlocks/correctness issues if a pgroup requires adjacent seats atomically. Coach-level partitioning is often the optimal trade-off.

# System Resilience Matrix

| Component        | Failure Mode       | Architectural Response                                                                                          |
|------------------|--------------------|-----------------------------------------------------------------------------------------------------------------|
| Redis            | Unavailable        | <b>Fail closed</b> for critical coordination, or rely on DB correctness boundary (accepting slower throughput). |
| SQL Database     | Unavailable        | <b>Retry carefully</b> with idempotency. Do not duplicate reservations.                                         |
| Kafka            | Unavailable        | <b>Outbox pattern</b> retains event in the DB until broker recovers. No data loss.                              |
| Application Node | Crashes after HOLD | <b>Background expiry/recovery job</b> identifies orphaned holds and releases inventory.                         |
| Payment Provider | Timeout            | <b>Treat as UNKNOWN.</b> Query provider actively using idempotency key.                                         |



# Operational Readiness: Observability

## Metrics (Business Health)

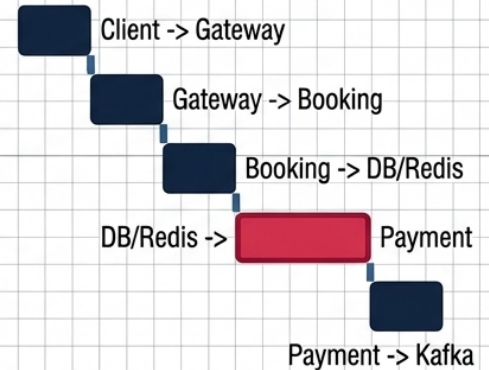


- booking success/failure rate
- reservation hold expiry rate
- payment timeout rate
- Redis lock contention
- Kafka consumer lag

## Logs (Context)

```
{
  'timestamp': '2023-10-27T10:15:30Z',
  'level': 'INFO',
  'message': 'Booking initiated',
  'trace_id': '...',
  'reservation_id': '...',
  'payment_id': '...',
  'event_id': '...'
}
```

## Distributed Tracing (Latency)



*“Observability is not just CPU usage; it is measuring the **health** of the business invariants.”*

# Defense in Depth: Enterprise Security

## Perimeter (API Gateway)

Authentication, Authorization, Rate-limiting high-risk APIs (e.g., Tatkai burst traffic).

## Transit & Storage

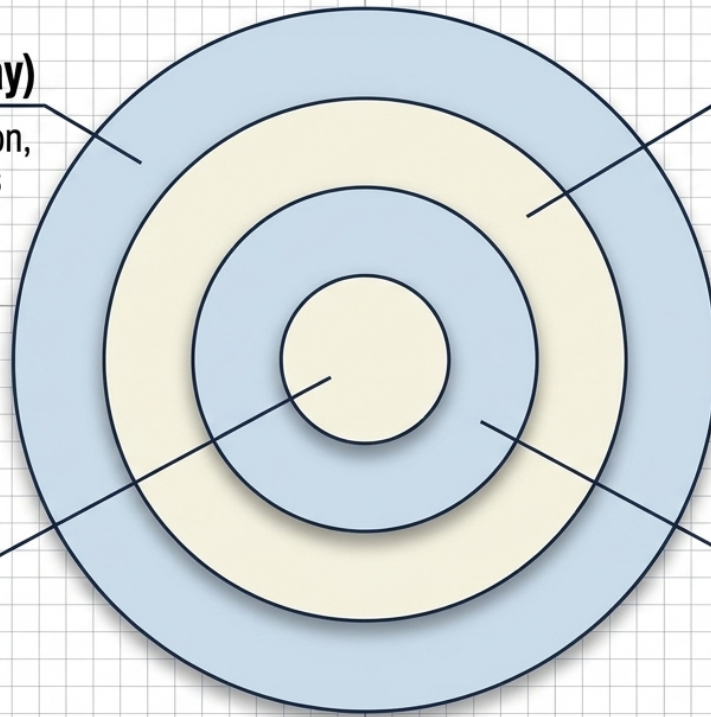
TLS in transit, encryption at rest for PII. Avoid storing raw card data (PCI compliance).

## Infrastructure

Least-privilege service credentials, strict secrets management.

## Application & Data

Validate passenger input, maintain immutable audit trails in Kafka.



# The Trinity of Distributed Booking

## **COORDINATE (Redis)**

Accelerates throughput and reduces distributed contention. The fast-path.

## **ENFORCE (SQL DB)**

The ultimate boundary of truth. Protects the booking invariant via ACID semantics. The safety net.

**High  
Throughput  
+  
Strict  
Correctness**

## **DECOUPLE (Kafka)**

Handles the asynchronous aftermath. Frees the write-path from external latencies.



# Six Core Architectural Principles



**Correctness First:** The database protects the invariant.



**Coordination:** Redis reduces contention, but is not a substitute for transactions.



**Idempotency:** Every retryable operation requires a safe strategy.



**Asynchronous Boundaries:** Kafka decouples non-critical downstream work.



**Outbox:** Use when state change and event must not diverge.



**Scale by Partitioning:** Avoid global locks and hot keys.